

PROJECT REPORT

CampusFlow: Smart Campus Resource Allocation & Event Management Engine

1. Cover Page

- Student Name: M T Akash Ayyappa
 - Registration Number: 24BCY10020
 - Course Title: Java Programming / Object-Oriented Software Engineering
 - Evaluation Type: Flipped Course Project Evaluation
 - Project Title: CampusFlow: Smart Campus Resource Allocation & Event Management Engine
 - Submission Format: Public GitHub Repository & Technical PDF Report
 - Target Domain: Core & Advanced Java / Academic Infrastructure Management
 - Date of Submission: September 2026
 - Academic Platform: VITyarthi Learning Destination
-

2. Introduction

Modern academic institutions manage expansive physical infrastructure to support educational, cultural, research, and sporting engagements. As student enrollment and extracurricular activities expand, coordinating access to high-demand spaces—such as central auditoriums, multimedia seminar halls, high-performance computing laboratories, and athletic complexes—becomes an increasingly complex logistical challenge.

Traditionally, universities rely on decentralized communication channels: paper booking forms, department emails, or generic spreadsheet logs. These manual mechanisms lack real-time synchronization, resulting in scheduling conflicts, under-utilized venues, administrative bottlenecks, and fire-safety compliance risks due to unmonitored event attendance.

CampusFlow was conceived as an automated, terminal-executable Java system engineered to address these challenges. By grounding its architecture in core Object-Oriented Programming (OOP) principles, algorithmic interval overlap detection, design patterns (Strategy, Repository, State Machine), and robust Role-Based Access Control (RBAC), CampusFlow delivers a deterministic, reliable, and auditable solution for campus facility governance.

3. Problem Statement

Manual and ad-hoc campus facility reservation workflows introduce five primary operational failure modes:

1. Time-Slot Collisions (Double Bookings): Lack of continuous mathematical conflict detection allows multiple departments or clubs to book the same facility during overlapping intervals.
2. Arbitrary Resource Allocation: When multiple groups seek the same physical venue simultaneously, decisions are made subjectively without formalized priority rules or policy-based scheduling algorithms.
3. Disconnection Between Booking and Public Event Attendance: Facilities are

booked without binding them to participant capacity, leading to unmonitored crowding and safety non-compliance.

4. Administrative Overhead & Lack of Visibility: Estate administrators spend hours reconciling requests manually and lack automated metrics on facility utilization rates or maintenance schedules.

5. Absence of Tamper-Evident Audit Trails: Booking modifications, cancellations, and approvals are untracked, preventing retrospective accountability.

CampusFlow directly resolves these problems by providing an automated, command-line executable Java engine equipped with interval mathematics, strategy-driven priority scoring, capacity-bounded event registration, and an immutable audit log.

4. Functional Requirements

Module 1: Identity & Role-Based Access Control (RBAC)

- User Authentication: Multi-role credential validation (Admin, Faculty, Student) with secure session state.
- Privilege Separation:
 - Administrator: Full authority to inspect all bookings, approve/reject pending student requests, toggle maintenance modes, and export audit trails.
 - Faculty: Immediate auto-approval for academic sessions, access to faculty-priority scheduling, and event sponsorship.
 - Student: Ability to browse available facilities, submit reservation requests to the administrative queue, and RSVP for published events.

Module 2: Resource Inventory & Facility Governance

- Facility Categorization: Structured catalog supporting AUDITORIUM, SEMINAR_HALL, COMPUTER_LAB, SMART_CLASSROOM, and SPORTS_COMPLEX.
- Dynamic Search & Filtering: Filter spaces by resource type, minimum capacity requirements, and operational status.
- Maintenance State Controls: Instant toggling of facility availability with mandatory justification logging, preventing reservations during repair windows.

Module 3: Scheduling Engine & Conflict Detection

- TimeSlot Interval Mathematics: Validation of reservation windows ($\text{startTime} < \text{endTime}$) and mathematical interval intersection detection across identical dates.
- Strategy-Based Priority Scoring: Computation of dynamic priority weights based on applicant role, duration, and academic purpose using the Strategy Pattern (PriorityAllocationStrategy).
- State Machine Lifecycle: Deterministic progression through PENDING, CONFIRMED, REJECTED, CANCELLED, and COMPLETED.

Module 4: Campus Event Lifecycle & Attendee RSVP Ticketing

- Booking-to-Event Association: Enforces that an event can only be scheduled against a CONFIRMED facility reservation.
- Capacity Boundary Enforcement: Hard validation ensuring maximum event attendee capacity never exceeds the host facility's physical seating limit.
- Thread-Safe Attendee Roster: Synchronized RSVP registration and cancellation preventing overselling.

Module 5: Operational Analytics & Audit Persistence

- Executive Utilization Metrics: Calculation of active vs. maintenance facilities, booking counts, approval rates, and highest-demand facility hotspots.

- Snapshot Export: Export of structured state snapshots and chronological audit logs to local text storage.
-

5. Non-Functional Requirements

1. Performance & Algorithmic Efficiency:
 - In-memory lookups operate in $O(1)$ time via `ConcurrentHashMap`.
 - Time-slot conflict detection evaluates interval overlaps mathematically in $O(1)$ per booking, scanning only active bookings.
 2. Security & Data Integrity:
 - Strict encapsulation preventing direct state tampering.
 - Granular RBAC checks protecting administrative endpoints.
 - Defensive copying of collections to preserve internal entity invariance.
 3. Reliability & Fault Tolerance:
 - Domain-specific checked exception hierarchy (`CampusFlowException`, `ResourceConflictException`, `ValidationException`, `UnauthorizedException`).
 - Graceful recovery from malformed terminal input without crashes.
 4. Maintainability & Modularity:
 - Adheres to SOLID architectural principles.
 - Clear decoupling across Model, Service, Strategy, Repository, and CLI presentation layers.
 5. Portability & Zero External Dependencies:
 - Executes across any standard Java SE runtime (Java 8 through Java 21) without requiring third-party runtime JARs.
 - Supports both interactive CLI and non-interactive automated test execution (`--demo`).
-

6. System Architecture

CampusFlow follows a classic Layered Architecture with unidirectional dependency flow:

```
...
+-----+
| PRESENTATION LAYER |
| CampusFlowApp | CliMenu | ConsoleColors |
+-----+
|
v
+-----+
| SERVICE LAYER |
| AuthService | FacilityService | BookingService |
| EventService | AnalyticsService |
+-----+
||
v v
+-----+ +-----+
| STRATEGY LAYER || DOMAIN MODEL LAYER |
| PriorityAllocationStrategy || User, Facility, TimeSlot, |
| StandardAllocationStrategy || Booking, CampusEvent, |
| FacultyPriorityStrategy || Role, ResourceType, Status |
+-----+ +-----+
\ /
\ /
```

```
v v
+-----+
| PERSISTENCE LAYER |
| DataStore (In-Memory) | PersistenceManager |
+-----+
...

```

7. Design Diagrams

7.1 Use Case Diagram (Textual Representation)

```
...
+-----+
| CampusFlow System |
| |
| [Student] ----> (Browse Facilities) |
| ----> (Request Facility Reservation) |
| ----> (View Personal Bookings) |
| ----> (RSVP / Ticketing for Campus Events) |
| |
| [Faculty] ----> (Request Auto-Approved Academic Booking) |
| ----> (Create & Publish Campus Event) |
| |
| [Admin] ----> (Review Pending Reservation Queue) |
| ----> (Approve / Reject Reservation) |
| ----> (Toggle Facility Maintenance State) |
| ----> (View System Utilization Analytics) |
| ----> (Export System Audit Snapshot) |
+-----+
...

```

7.2 Process Flow / Workflow Diagram

```
...
[Start Booking Request]
|
v
[Select Facility & TimeSlot (Date, Start, End)]
|
v
<Is Facility in Maintenance?> ---- YES ----> [Throw ResourceConflictException
(Rejected)]
|
NO
v
<Does Slot Overlap with Confirmed/Pending Booking?> -- YES --> [Throw
ResourceConflictException (Rejected)]
|
NO
v
[Compute Priority Score via Strategy Pattern]
|
v
<Is Requester Faculty or Admin?>
| |
YES NO

```

```
||
v v
[Status: CONFIRMED] [Status: PENDING]
(Auto-Approved) (Queued for Admin Review)
||
+-----+-----+
|
v
[Save to DataStore]
|
v
[Append to AuditLog]
|
v
[End]
...

```

7.3 Sequence Diagram: Admin Booking Approval

```
...
AdminUser CliMenu BookingService DataStore AuditLog
|||||
|--Select Req #-->|||
| |--approveBooking()->||
|| |--findBooking()--->|
|| |<--Return Booking--|
|||||
|| | |--Set CONFIRMED--->|
|| | |--addAuditLog()----->|
|| |<--Success Return--||
|<--Confirm Msg---|||
...

```

7.4 Class / Component Diagram

```
...
+-----+ +-----+
| TimeSlot || Booking |
+-----+ +-----+
| - date: LocalDate || - id: String |
| - startTime: LocalTime || - facilityId: String |
| - endTime: LocalTime || - requestedByUserId: String |
+-----+ | - timeSlot: TimeSlot |
| + overlapsWith(TimeSlot) |<-----| - status: BookingStatus |
| + getDurationMinutes() || - priorityWeight: int |
+-----+ +-----+
|
+-----+ |
| Facility ||
+-----+ v
| - id: String | +-----+
| - name: String || CampusEvent |
| - type: ResourceType | +-----+
| - capacity: int || - eventId: String |
| - underMaintenance: bool || - bookingId: String |
+-----+ | - maxAttendees: int |
| - registeredAttendees: Set<String> |
+-----+ +-----+
| User || + registerAttendee(studentId): bool |

```

```
+-----+ +-----+
| - id: String |
| - role: Role |
| - department: String |
+-----+
...
```

7.5 Entity-Relationship (ER) Schema Design

```
...
+-----+ 1:N +-----+
| USER |-----< | BOOKING |
+-----+ +-----+
| PK id | | PK id |
| fullName | | FK requestedBy |
| email | | FK facilityId |
| role (ENUM) | | date |
| department | | startTime |
+-----+ | endTime |
| | status (ENUM) |
| | priorityScore |
| +-----+
| | 1:1
| |
| 1:N +-----+
+-----< | CAMPUS_EVENT |
(RSVP) +-----+
| PK eventId |
| FK bookingId |
| FK organizerId |
| title |
| maxAttendees |
+-----+
...
```

8. Design Decisions & Rationale

1. Pure Java Standard Library Core (Zero External Runtime Dependencies):
 - Decision: Avoided heavyweight external frameworks (Spring Boot, Hibernate) in the runtime execution path.
 - Rationale: Ensures the evaluator's automated test harness can compile and execute the system seamlessly on any terminal environment using standard javac and java without requiring dependency resolution or internet access.
2. Strategy Pattern for Priority Scheduling:
 - Decision: Extracted scoring logic into PriorityAllocationStrategy implemented by StandardAllocationStrategy and FacultyPriorityAllocationStrategy.
 - Rationale: Enables institutions to dynamically alter allocation policies (e.g., peak exam hours vs. cultural festival season) without modifying the core booking engine, complying with the Open/Closed Principle.
3. Dual Execution Mode (Interactive CLI + --demo Headless Pipeline):
 - Decision: Supported both interactive console menus and automated script execution via --demo.
 - Rationale: Ensures human evaluators can manually interact with the application, while automated grading pipelines can run non-interactively and verify all requirements in under 3 seconds.
4. Thread-Safe Concurrency Primitives:

- Decision: Used ConcurrentHashMap, CopyOnWriteArrayList, and synchronized methods for RSVP ticketing and booking requests.
 - Rationale: Eliminates race conditions during concurrent event reservations and guarantees data integrity.
-

9. Implementation Details

- Package Hierarchy: Clean layered structure (com.campusflow.model, com.campusflow.service, com.campusflow.strategy, com.campusflow.repository, com.campusflow.exception, com.campusflow.ui).
 - Interval Overlap Logic: Encapsulated in TimeSlot.overlapsWith():
$$\text{Overlap} = (\text{date}_1 == \text{date}_2) \wedge (\text{start}_1 < \text{end}_2) \wedge (\text{end}_1 > \text{start}_2)$$
 - Custom Exceptions: Structured hierarchy inheriting from CampusFlowException providing detailed diagnostic field metadata.
 - Audit Logging: Thread-safe append-only ledger tracking all administrative actions, logins, state transitions, and conflict events.
-

10. Screenshots & Execution Results

The application includes an automated demonstration runner (./run.sh --demo) that executes all modules sequentially:

...

[STEP 1] Testing Authentication & RBAC

- > Admin authenticated: Dr. Rajesh Sharma (ADM01) - System Administrator [Estate Office]
- > Student authenticated: Aarav Patel (STU01) - Student [Computer Science]
- > Authentication & Session Management: PASSED

[STEP 2] Querying Facility Inventory

- > Total facilities registered: 5
- [FAC-AUD-01] Sarojini Naidu Central Auditorium (Auditorium, Cap: 600, Loc: Main Campus Block A)
- [FAC-SEM-02] Aryabhata Seminar Hall (Seminar Hall, Cap: 120, Loc: Science Block B)
- [FAC-LAB-03] Alan Turing Computing Lab (Computer Lab, Cap: 75, Loc: Tech Tower C)
- [FAC-CLS-04] Smart Classroom 301 (Smart Classroom, Cap: 90, Loc: Academic Wing D)
- [FAC-SPT-05] Indoor Badminton Arena (Sports Complex, Cap: 150, Loc: Student Activity Center)
- > Facility Inventory Discovery: PASSED

[STEP 3] Booking Creation & Priority Calculation

- > Student created reservation: Booking #BKG-9A2E1F | Facility: FAC-SEM-02 | User: STU01 | Slot: 2026-09-21 [14:00 - 17:00] | Status: PENDING | Priority: 10
- > Booking Request Workflow: PASSED

[STEP 4] Verifying Conflict Detection Engine

- > Attempting overlapping reservation on FAC-SEM-02 at 2026-09-21 [15:00 - 16:00]...
- > SUCCESS: Conflict successfully detected and rejected!
- Caught Exception: Time-slot conflict detected with Booking [BKG-9A2E1F] by User [STU01]
- > Conflict Detection Engine: PASSED

[STEP 5] Admin Review & Approval Queue

-> Admin approved booking BKG-9A2E1F
-> Updated Status: CONFIRMED (Approved by Admin Dr. Rajesh Sharma)
-> Approval Workflow: PASSED

[STEP 6] Event Lifecycle & RSVP Ticketing

-> Event Created: [EVT-C814D0] VIT Robotics Hackathon 2026 | Org: STU01 | Seats: 0/100 | Published: true
-> Attendee Diya Mehta RSVP status: SUCCESS
-> Available Seats: 99 / 100
-> Event Ticketing & Capacity: PASSED

[STEP 7] Generating Real-Time System Analytics

Total Facilities Managed : 5
Operational Facilities : 5
Confirmed Active Bookings : 2
System Booking Approval Rate : 100.0%
-> Analytics Calculation: PASSED

[STEP 8] Verifying Persistence & Audit Snapshot

-> Audit report written to: ./data/campusflow_audit_report.txt (Size: 2180 bytes)
-> State Persistence & Audit Logging: PASSED
...

11. Testing Approach

Testing is implemented at two distinct levels:

1. JUnit 5 Test Suite (BookingServiceTest.java): Standard automated assertions for Maven test workflows.
2. Standalone Zero-Dependency Test Harness (StandaloneTestRunner.java): Executes pure Java assertions directly via javac without requiring external libraries.

Test Matrix:

Test ID	Test Case Description	Target Component	Expected Result	Status
TC-01	Interval overlap boundary check	TimeSlot	Overlap detected on intersecting windows; adjacent windows allowed	PASS
TC-02	Valid credential login	AuthService	Successfully authenticates user and issues session	PASS
TC-03	Invalid password rejection	AuthService	Throws UnauthorizedException	PASS
TC-04	Duplicate facility ID guard	FacilityService	Throws ValidationException on existing ID	PASS
TC-05	Double-booking clash detection	BookingService	Throws ResourceConflictException	PASS
TC-06	Student request lifecycle	BookingService	Initial state PENDING, updates to CONFIRMED on admin approval	PASS
TC-07	Event capacity overflow guard	EventService	Throws ValidationException when event capacity exceeds facility	PASS

12. Challenges Faced

1. Interval Overlap Boundary Edge Cases: Defining precise mathematical conditions for contiguous slots (e.g., Slot 1 ending at 14:00 and Slot 2 starting at 14:00). Resolved by defining half-open interval checks where end-point adjacency does not trigger a collision.

2. Deterministic Evaluation in Headless Environments: Automated grading environments often fail when waiting for interactive console input. Resolved by engineering the --demo execution flag, allowing the grader to run the entire verification suite non-interactively.
 3. Decoupled Event Capacity Constraints: Ensuring event creation guarantees attendee counts do not breach physical venue capacity required cross-service validation linking EventService, BookingService, and FacilityService.
-

13. Learnings & Key Takeaways

- Mastery of Java Object-Oriented paradigms (polymorphism, abstraction, encapsulation).
 - Implementation of the Strategy Pattern for algorithmic business rules.
 - Practical experience with concurrent collections and thread-safe synchronization.
 - Designing zero-dependency software architecture that runs reliably on diverse terminal environments.
-

14. Future Enhancements

- Integration with campus IoT hardware (smart electronic door locks and biometric badge scanners).
 - Real-time notification services via SMS/Email (Twilio / SendGrid integration).
 - Relational database integration (PostgreSQL / JDBC connection pooling) for multi-year historical storage.
 - RESTful API layer and web/mobile dashboards using Spring Boot and React.
-

15. References

1. Bloch, Joshua. Effective Java (3rd Edition). Addison-Wesley Professional, 2018.
2. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
3. Oracle Java Documentation: Java Platform, Standard Edition 8 / 11 / 17 API Specification.
4. VITyarthi Project Guidelines & Rubrics Document: BuildYourOwnProjectVITyarthi.pdf.